COMP438/538 Reinforcement Learning

Model-Free Prediction

Barış Akgün

Fall 2026

Models and Prediction without the Models

So far:

- MDPs
  - State, Action, Reward, Transition Dynamics, Discount
  - Optionally terminal and start states
- Returns, Values and Q-Values

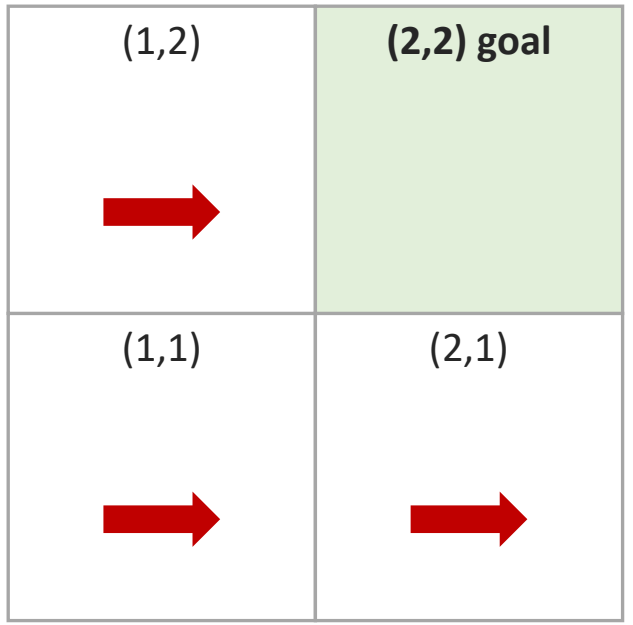
- Policy Evaluation
- Policy Iteration (Evaluation then Selection)
- Value Iteration (Concurrent Evaluation and Selection)

- Which computations need the transition and reward model?
- Which action-selection step can use a learned table directly?

Policy evaluation without the model

- With a model: evaluate a policy, then improve it.
- Today: evaluate the **same fixed policy** using observed experience.
- Next week: improve policies using experience.

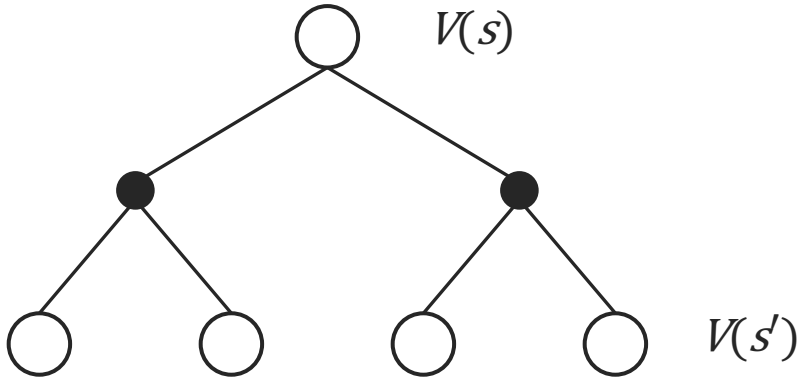
Question: You can run this policy and observe what happens. How could you estimate $v^{\pi}((1,1))$?

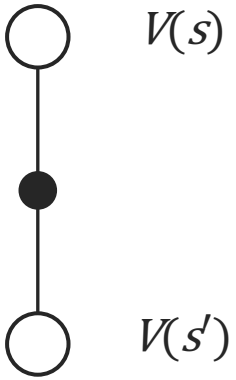


Always choose right; γ=0.5.

Transition probabilities: **?**

Why learn from samples?





Expected backup	Sample backup
Average over possible outcomes using the model	Use an observed transition
Need outcome probabilities and expected rewards	Receive $(S_t, A_t, R_{t+1}, S_{t+1})$

- A robot's dynamics may be unknown or unreliable.
- A simulator may produce samples even when enumerating all outcomes is impractical.
- One cheap update can still require many samples to learn accurately.

Reinforcement Learning: Models are unknown ahead of time (Today: Sampling, Later: Model-Learning)

Side Note: Three different questions about the data

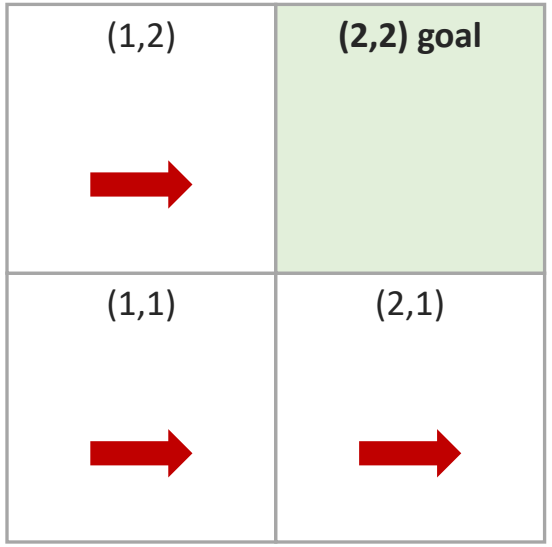
Question	Distinction
Do we use or learn an environment model?	Model-based / model-free
Can we collect more interaction data while learning?	Online / offline (fixed data)
Does the behaviour generating data match the policy being evaluated?	On-policy / off-policy

Today: tabular, model-free, on-policy prediction of a fixed π.

One observed episode

(1,1) → (1,1) → (1,2) → goal

t	S_t	A_t	R_{t+1}	S_{t+1}
0	(1,1)	right	−0.04	(1,1)
1	(1,1)	right	−0.04	(1,2)
2	(1,2)	right	+1	goal



Important concepts:

- Transitions
- Trajectories/Roll-outs
- Episode
- Return
- Averaging Returns

Monte Carlo: value as a mean return

$$v^{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid S_t = s]$$

Monte Carlo prediction: follow π, observe complete episodes, and average returns after visits to each state.

$$G_T = 0, \qquad G_t = R_{t+1} + \gamma G_{t+1}$$

Episode: (1,1) → (1,1) → (1,2) → goal; rewards −0.04, −0.04, +1; $\gamma = 0.5$.

Your turn: calculate G_2, then G_1, then G_0 for our episode.

$$G_2 = 1, \qquad G_1 = -0.04 + 0.5(1) = 0.46, \qquad G_0 = -0.04 + 0.5(0.46) = 0.19$$

One observed return is a sample, not the expectation.

Which visits count?

Visit to (1,1)	Return following that visit
First, t=0	0.19
Second, t=1	0.46

Episode: (1,1) → (1,1) → (1,2) → goal

- **First-visit MC:** include only the first visit to a state in each episode.
- **Every-visit MC:** include every visit to that state.

Question: what sample mean does each produce after this one episode? (For (1,1))

Answer: first-visit 0.19; every-visit $(0.19 + 0.46)/2 = 0.325$.

From an average to an incremental update

For one newly included return G_t, first increment $N(s)$:

$$N(s) \leftarrow N(s) + 1$$

$$V(s) \leftarrow \frac{(N(s) - 1)V(s) + G_t}{N(s)} = V(s) + \frac{1}{N(s)}[G_t - V(s)]$$

A common update form

$$V(S_t) \leftarrow V(S_t) + \alpha[U_t - V(S_t)]$$

U_t: target. α: fraction of the discrepancy used for this update.

Step size for MC	Meaning
$1/N(s)^*$	Exact sample mean of included returns
Constant α	Recency weighting; initialization can still matter, better for non-stationary problems

*This is a special case of decayed α that meets the Robbins–Monro conditions for convergence, but not all decay schedules work

First-visit MC prediction

1. Fix π. Initialize $V(s) = 0$ and $N(s) = 0$ for nonterminal states.
2. Generate a complete episode following π.
3. Compute its returns backwards: $G_T = 0$, then $G_t = R_{t+1} + \gamma G_{t+1}$.
4. Scan the episode in time order. At the **first visit** to each state:
   - increment $N(S_t)$;
   - update $V(S_t) \leftarrow V(S_t) + \frac{1}{N(S_t)}[G_t - V(S_t)]$.
5. Repeat with another episode.

For constant-step MC, replace $1/N(S_t)$ by α in step 4. The first-visit rule stays.

Question: How do the steps change if we do the every-visit version?

Driving home: must learning wait?

Predict the minutes remaining until home

Situation	Elapsed minutes	Predicted minutes remaining	Predicted total minutes
Leave office	0	30	30
Reach car; raining	5	35	40
Exit highway	20	15	35
Behind slow truck	30	10	40
Enter home street	40	3	43
Arrive home	43	0	43

At the car, would you revise the office prediction now, or wait until home?

Another Question: What happens if the problem is continuing?

Standalone prediction example: rewards are elapsed minutes; $\gamma = 1$. Book Example 6.1 (Sutton & Barto, 2018).

A sampled Bellman target – Temporal Difference (TD) Learning

1. Recall the Bellman expectation equation:

$$v^{\pi}(s) = \mathbb{E}_{\pi}[R_{t+1} + \gamma v^{\pi}(S_{t+1}) \mid S_t = s]$$

2. Use one observed transition and the current next-state estimate:

$$U_t = R_{t+1} + \gamma V(S_{t+1})$$

3. Compare the target with the current estimate, then update:

$$\delta_t = U_t - V(S_t), \qquad V(S_t) \leftarrow V(S_t) + \alpha\delta_t$$

Bootstrapping: use an existing value estimate in a new update target.

The expected TD target is a Bellman backup of the current V; it need not equal v^{π}.

TD(0) prediction

Fix π and choose α. Initialize the nonterminal estimates; set $V(\text{terminal}) = 0$.

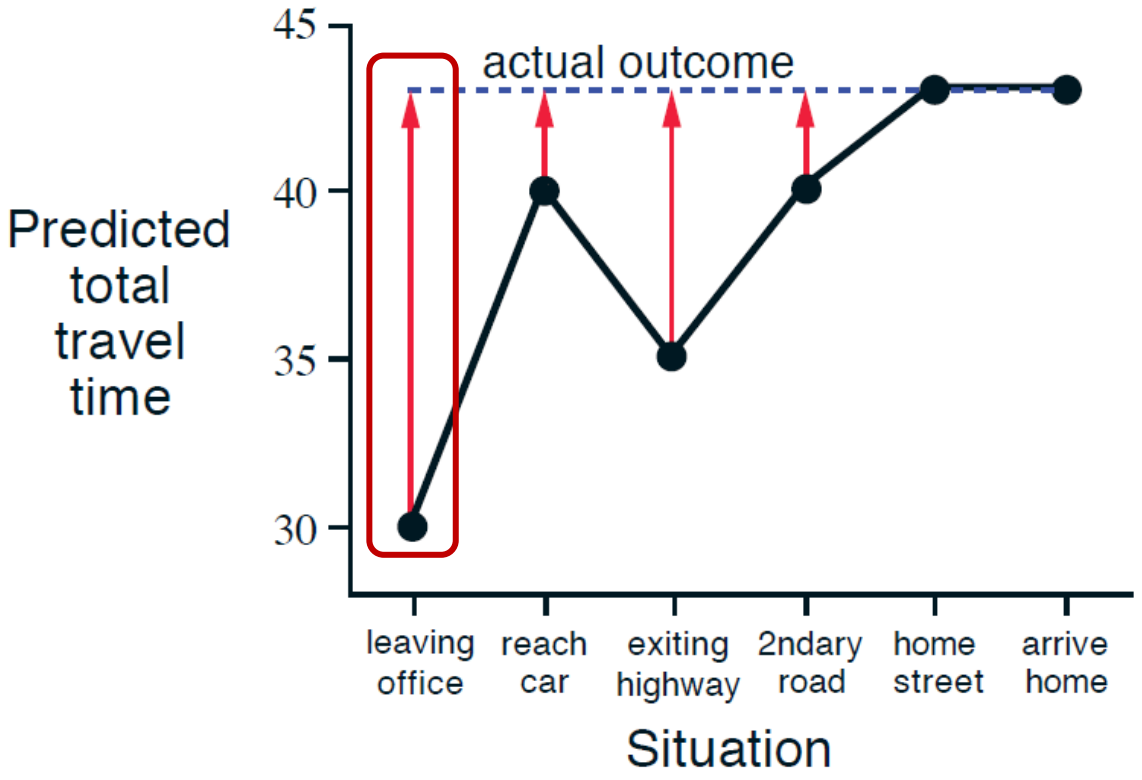
At each transition:

1. Take A_t according to π; observe R_{t+1} and S_{t+1}.
2. Form $U_t = R_{t+1} + \gamma V(S_{t+1})$.
3. Update $V(S_t) \leftarrow V(S_t) + \alpha[U_t - V(S_t)]$.
4. Advance to the next state; if terminal, end the episode.

Update after observing the transition. Read the current table before changing it.

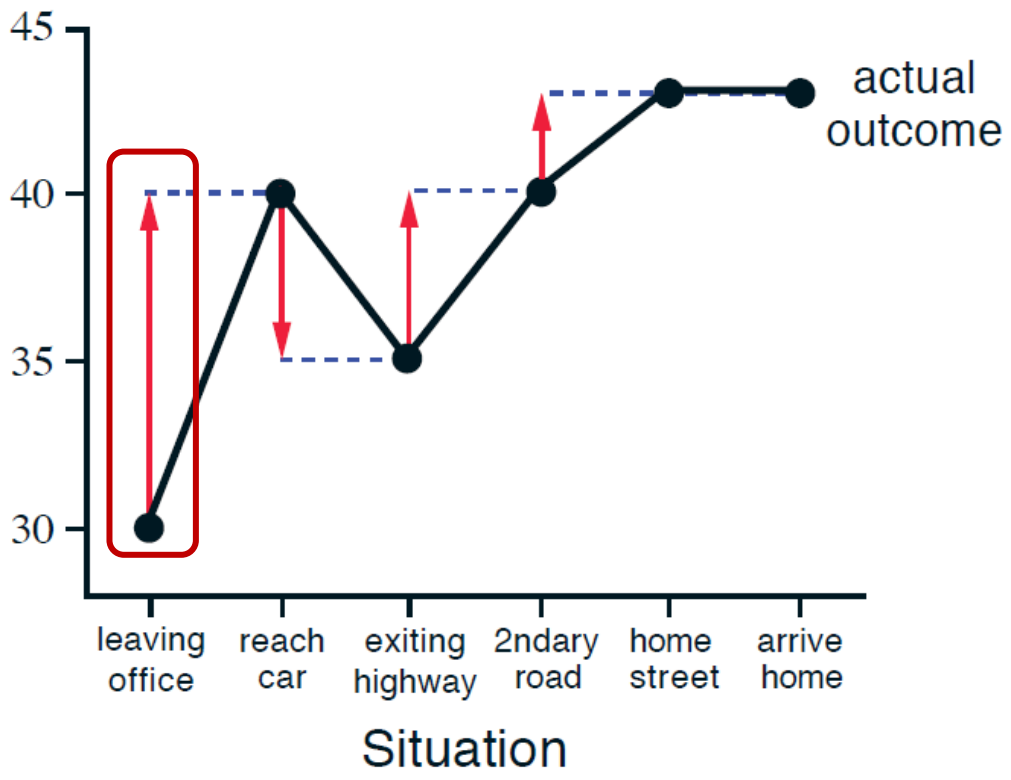
Driving home: what each update changes

MC: toward the final outcome



First arrow: office prediction $30 \to 43$, after reaching home

TD: toward the next prediction



First arrow: office prediction $30 \to 5+35=40$, after reaching the car

$\gamma=1$; $\alpha=1$ for the full arrows. Figure: Sutton & Barto (2018), Figure 6.1.

Your turn: MC and TD on the same episode

(1,1) → (1,1) → (1,2) → goal; rewards −0.04, −0.04, +1; returns 0.19, 0.46, 1

Reset each method independently: $V = 0$, $\gamma = 0.5$, **constant** $\alpha = 0.5$.

1. First-visit MC: update each state's first visit using its completed return.
2. TD(0): update after each transition, in time order.
3. Why does the goal reward affect the estimates differently?

Method / event	Target U_t	Estimate being changed	Value after update
MC, first (1,1) visit	0.19	$V((1,1))$	0.095
MC, first (1,2) visit	1	$V((1,2))$	0.5
TD, t=0	−0.04	$V((1,1))$	−0.02
TD, t=1	−0.04	$V((1,1))$	−0.03
TD, t=2	1	$V((1,2))$	0.5

MC and TD have different targets and update schedules; this is not a performance ranking.

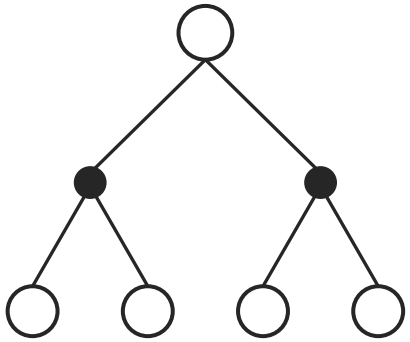
One episode versus an expected value

State	First-visit MC after this episode	TD(0) after this episode	True v^{π} from the known model
(1,1)	0.095	−0.03	≈ 0.051
(1,2)	0.5	0.5	≈ 0.836

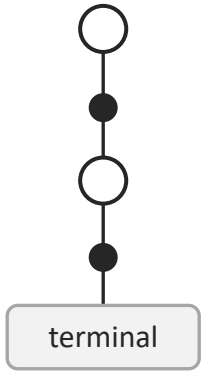
The right-hand column is a checking benchmark; the learner did not use it.

Questions: Can this episode establish which method is better? What would we need to compare learning after many episodes?

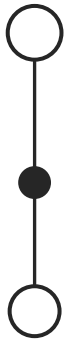
Sampling and bootstrapping



DP: all successors, one step



MC: one path to termination



TD(0): one sampled step

Method	Uses sampled experience?	Bootstraps from value estimates?	What supplies the target?
DP expected evaluation backup	No	Yes	Model-weighted reward and successor values
Complete-return MC	Yes	No	Observed return through termination
TD(0)	Yes	Yes	One observed reward and next-state estimate

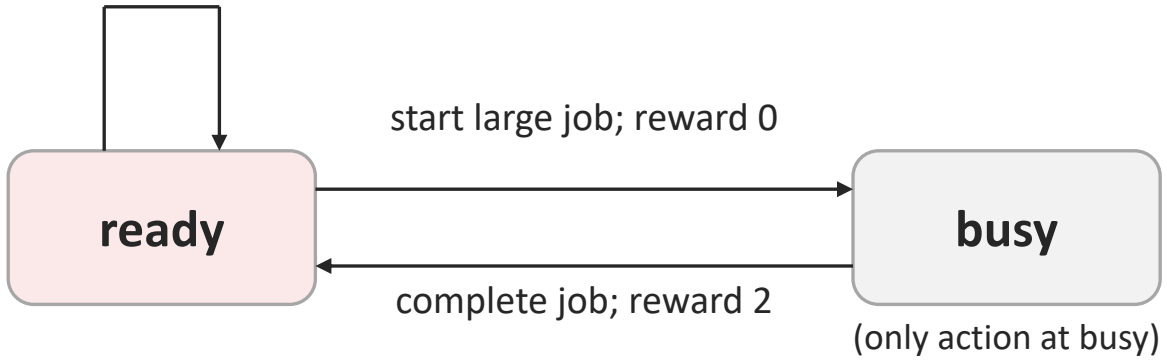
Sampling: use observed outcomes in place of a full expectation. **Bootstrapping:** use a value estimate in the target.

Check: Which needs a model? Which must wait for the completed return? Which can update before the episode ends?

A task that keeps running

A continuing service system — equal-duration time steps

steady service; reward 0.8



Starting from ready:

Fixed policy at ready	Reward sequence
Always steady service	0.8, 0.8, 0.8, …
Always start a large job	0, 2, 0, 2, …

There is no terminal state. What should good long-run performance mean?

Discounted return and average reward

Average reward: expected reward per step over a long run

For the policies in this service example, starting from ready:

$$r(\pi) \doteq \lim_{t\to\infty} \frac{1}{t} \mathbb{E}_\pi[R_1 + R_2 + \cdots + R_t \mid S_0 = \text{ready}]$$

Policy	Average reward per step	Discounted return from ready, γ=0.5
Steady service	0.8	1.6
Large jobs	(0 + 2)/2 = 1	4/3 ≈ 1.333

The objective changes the ranking: discounted return prefers steady service; average reward prefers large jobs.

Self-Study Suggestion: Sutton & Barto §10.3 for the differential return and values that average-reward methods learn.

What can we learn before the episode ends?

- MC updates toward a completed observed return.
- TD updates toward an observed reward plus estimated continuation.
- Both can evaluate a fixed policy from experience.

Which needs the next state's estimate? Which needs the completed return?

Next session: bias, variance, n-step targets and evaluating learning.

Next topic: improve policies using learned action values.